

**Digital Nurture 5.0 Deep Skilling Handbook Java FSE -
Solution**

PL SQL Programming

Supersetid :8003253

Table creation:

```
CREATE TABLE Customers (  
  CustomerID NUMBER PRIMARY KEY,  
  Name VARCHAR2(100),  
  DOB DATE,  
  Balance NUMBER,  
  LastModified DATE  
);
```

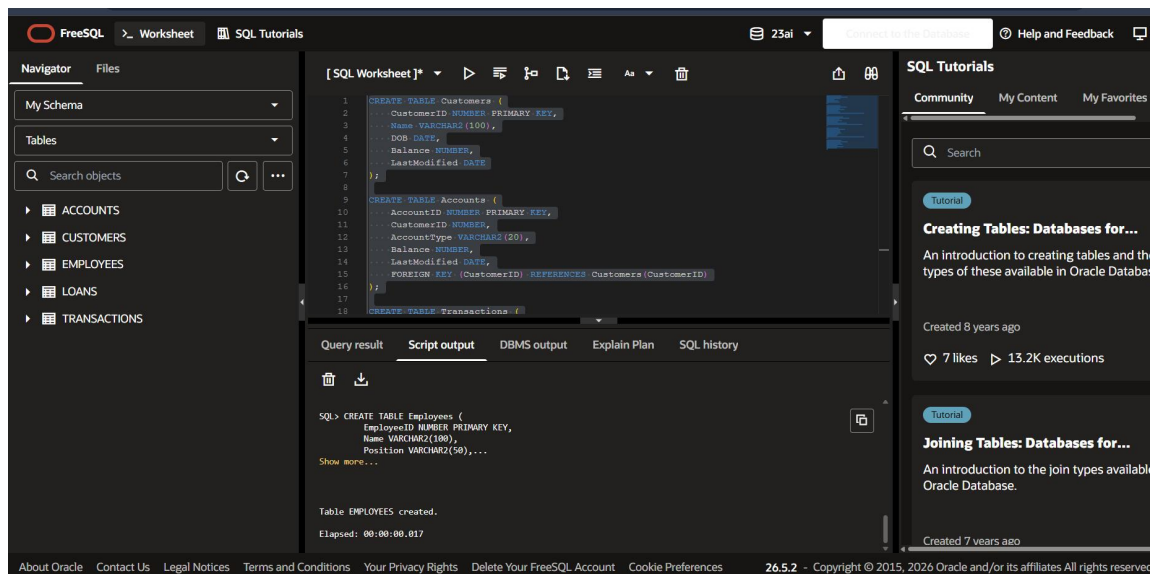
```
CREATE TABLE Accounts (  
  AccountID NUMBER PRIMARY KEY,  
  CustomerID NUMBER,  
  AccountType VARCHAR2(20),  
  Balance NUMBER,  
  LastModified DATE,  
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Transactions (  
  TransactionID NUMBER PRIMARY KEY,  
  AccountID NUMBER,  
  TransactionDate DATE,  
  Amount NUMBER,  
  TransactionType VARCHAR2(10),  
  FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);
```

```
CREATE TABLE Loans (  
  LoanID NUMBER PRIMARY KEY,  
  CustomerID NUMBER,  
  LoanAmount NUMBER,  
  InterestRate NUMBER,  
  StartDate DATE,  
  EndDate DATE,  
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Employees (  
  EmployeeID NUMBER PRIMARY KEY,  
  Name VARCHAR2(100),  
  Position VARCHAR2(50),  
  Salary NUMBER,  
  Department VARCHAR2(50),  
  HireDate DATE
```

);



Value Insertion:

INSERT INTO Customers

VALUES (1,'John Doe',TO_DATE('1985-05-15','YYYY-MM-DD'),1000,SYSDATE);

INSERT INTO Customers

VALUES (2,'Jane Smith',TO_DATE('1990-07-20','YYYY-MM-DD'),1500,SYSDATE);

INSERT INTO Accounts

VALUES (1,1,'Savings',1000,SYSDATE);

INSERT INTO Accounts

VALUES (2,2,'Checking',1500,SYSDATE);

INSERT INTO Transactions

VALUES (1,1,SYSDATE,200,'Deposit');

INSERT INTO Transactions

VALUES (2,2,SYSDATE,300,'Withdrawal');

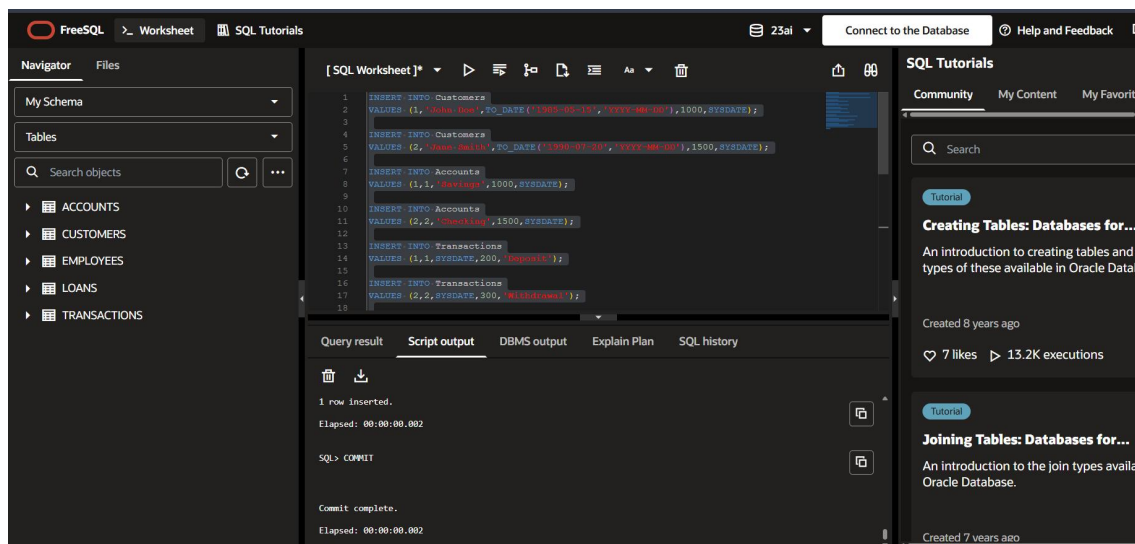
INSERT INTO Loans

VALUES (1,1,5000,5,SYSDATE,ADD_MONTHS(SYSDATE,60));

INSERT INTO Employees

VALUES (1,'Alice Johnson','Manager',70000,'HR',
TO_DATE('2015-06-15','YYYY-MM-DD'));

INSERT INTO Employees



```
VALUES (2,'Bob Brown','Developer',60000,'IT',
TO_DATE('2017-03-20','YYYY-MM-DD'));
```

Exercise 1: Control Structures

```
SET SERVEROUTPUT ON;
```

```
INSERT INTO Customers
```

```
(CustomerID, Name, DOB, Balance, LastModified, IsVIP)VALUES(3,'Ramesh',TO_DATE('1950-01-10','YYYY-MM-DD'),20000,SYSDATE,NULL);
```

```
INSERT INTO LoansVALUES(2,3,10000,8,SYSDATE,SYSDATE+20);
```

```
COMMIT;
```

```
-- Scenario 1
```

```
DECLARE
```

```
age NUMBER;
```

```
BEGIN
```

```
FOR c IN (SELECT * FROM Customers) LOOP
```

```
age := TRUNC(MONTHS_BETWEEN(SYSDATE,c.DOB)/12);
```

```
IF age > 60 THEN
```

```
UPDATE Loans
```

```
SET InterestRate = InterestRate - 1
```

```
WHERE CustomerID = c.CustomerID;
```

```
DBMS_OUTPUT.PUT_LINE('Discount Applied to ' || c.Name);
```

```
END IF;
```

```
END LOOP;
```

```
COMMIT;
```

```
END;
```

```
/
```

```
-- Scenario 2
```

```
BEGIN
```

```
UPDATE Customers
```

```
SET IsVIP='TRUE'
```

```
WHERE Balance>10000;
```

```
COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('VIP Status Updated');
```

```
END; /
```

Supersetid :8003253

-- Display VIP Customers

```
SELECT CustomerID,
```

```
    Name,
```

```
    Balance,
```

```
    IsVIP
```

```
FROM Customers;
```

-- Scenario 3

```
BEGIN
```

```
  FOR r IN (
```

```
    SELECT c.Name,
```

```
          l.EndDate
```

```
    FROM Customers c
```

```
    JOIN Loans l
```

```
    ON c.CustomerID=l.CustomerID
```

```
    WHERE l.EndDate<=SYSDATE+30)
```

```
  LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(
```

```
      'Reminder : '
```

```
      ||r.Name||
```

```
      ' Loan Due on '
```

```
      ||TO_CHAR(r.EndDate,'DD-MON-YYYY'));
```

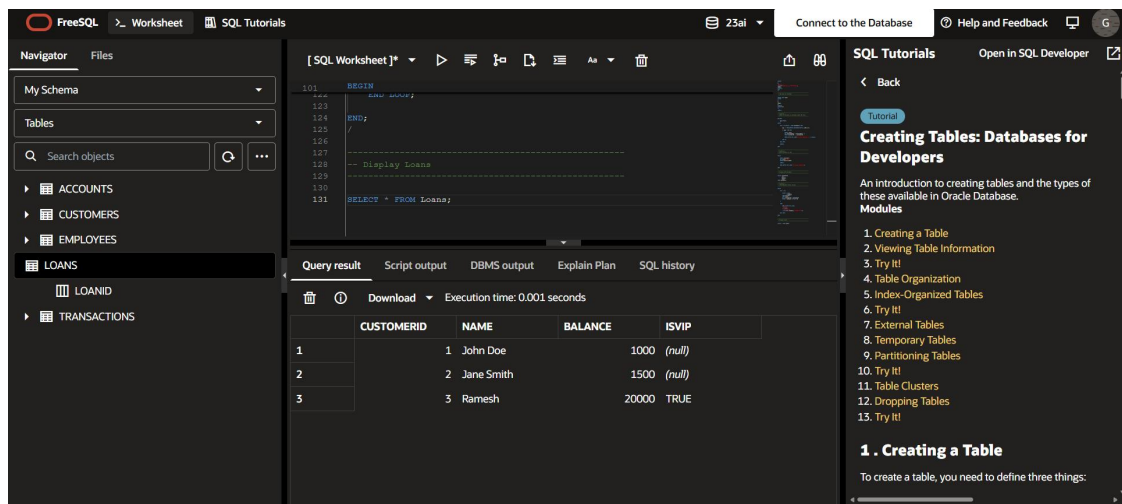
```
  END LOOP;
```

```
END;
```

```
/
```

```
SELECT * FROM Loans;
```

OUTPUT:



The screenshot displays the FreeSQL web application interface. On the left, a sidebar shows a database schema with tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, LOANID, and TRANSACTIONS. The main area shows a SQL worksheet with the following code:

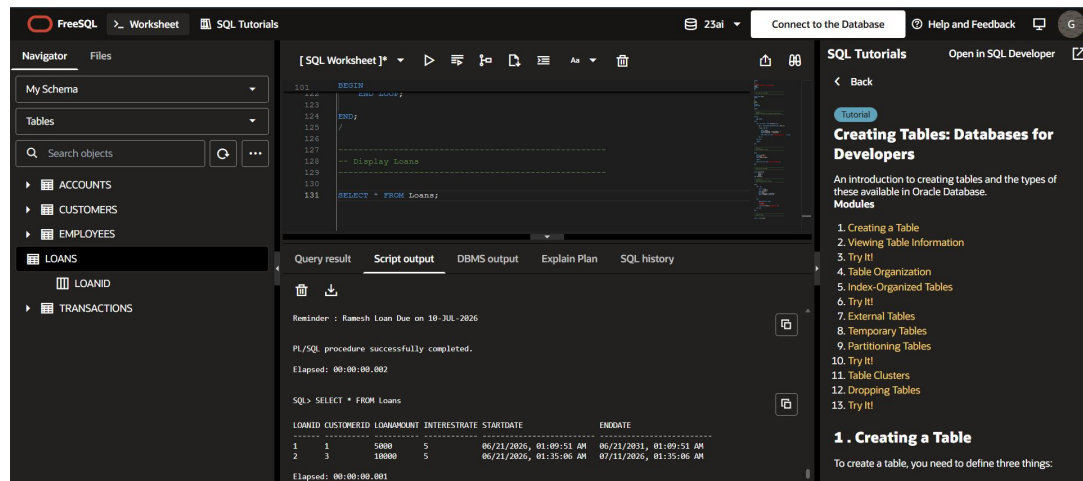
```
101 BEGIN
102     REMINDER;
103
104 END;
105 /
106
107 ----- Display Loans -----
108
109
110
111 SELECT * FROM Loans;
```

Below the code, the 'Query result' tab is active, showing the execution output. The execution time is 0.001 seconds. The result is a table with the following data:

	CUSTOMERID	NAME	BALANCE	ISVIP
1	1	John Doe	1000	(null)
2	2	Jane Smith	1500	(null)
3	3	Ramesh	20000	TRUE

On the right side of the interface, there is a 'SQL Tutorials' section titled 'Creating Tables: Databases for Developers'. It includes a list of modules and a section titled '1. Creating a Table'.

Supersetid :8003253



Exercise 2: Error Handling

SET SERVEROUTPUT ON;

-- Scenario 1

-- Safe Transfer Funds

CREATE OR REPLACE PROCEDURE SafeTransferFunds(

 fromAcc NUMBER,

 toAcc NUMBER,

 amount NUMBER

)

IS

 balance NUMBER;

BEGIN

 SELECT Balance

 INTO balance

 FROM Accounts

 WHERE AccountID = fromAcc;

 IF balance < amount THEN

 DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

 ROLLBACK;

 ELSE

 UPDATE Accounts

 SET Balance = Balance - amount

 WHERE AccountID = fromAcc;

 UPDATE Accounts

 SET Balance = Balance + amount

 WHERE AccountID = toAcc;

 COMMIT;

 DBMS_OUTPUT.PUT_LINE('Amount Transferred Successfully');

 END IF;

EXCEPTION

 WHEN OTHERS THEN

 ROLLBACK;

 DBMS_OUTPUT.PUT_LINE('Error : ' || SQLERRM);

Supersetid :8003253

```
END;
/
-- Test Scenario 1
BEGIN
    SafeTransferFunds(1,2,500);
END;
/
SELECT * FROM Accounts;
-- Scenario 2
-- Update Employee Salary
CREATE OR REPLACE PROCEDURE UpdateSalary(
    empId NUMBER,
    percent NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * percent / 100)
    WHERE EmployeeID = empId;
    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Employee Not Found');
    ELSE
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');
    END IF;
END;
/
-----
-- Test Scenario 2
-----

BEGIN
    UpdateSalary(1,10);
END;
/
SELECT * FROM Employees;
-- Scenario 3
-- Add New Customer
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    cid NUMBER,
    cname VARCHAR2,
    dob DATE,
    bal NUMBER
)
IS
BEGIN

    INSERT INTO Customers(CustomerID, Name, DOB, Balance, LastModified, IsVIP)VALUES(cid, cname, dob,
    bal, SYSDATE, NULL);
    COMMIT;
```

Supersetid :8003253

```
DBMS_OUTPUT.PUT_LINE('Customer Added Successfully');
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Customer ID Already Exists');
END;
/
-- Test Scenario 3
BEGIN
  AddNewCustomer(
    4,
    'Rahul',
    TO_DATE('1998-06-10','YYYY-MM-DD'),
    12000
  );
END;
/
-- Check Customers
SELECT * FROM Customers;
```

OUTPUT:

The screenshot shows the FreeSQL IDE interface. The left sidebar displays a database schema with tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, LOANID, and TRANSACTIONS. The main editor displays a SQL script with the following content:

```
109
110
111
112
113
114
115
116
117
118
119
120
121
```

The script includes a PL/SQL procedure call and a query:

```
-- Check Employees
SELECT * FROM Employees;
Scenario 3
-- Add New Customer
```

The output pane shows the results of the execution:

Customer Added Successfully

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010

SQL> SELECT * FROM Customers

CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	John Doe	05/15/1985, 05:30:00 AM	1000	06/21/2026, 01:09:50 AM	
2	Jane Smith	07/20/1990, 05:30:00 AM	1500	06/21/2026, 01:09:58 AM	
3	Rahul	05/20/1998, 05:30:00 AM	12000	06/21/2026, 01:47:15 AM	
4	Ramesh	01/10/1990, 05:30:00 AM	20000	06/21/2026, 01:34:52 AM	TRUE

The screenshot shows the FreeSQL IDE interface. The left sidebar displays a database schema with tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, LOANID, and TRANSACTIONS. The main editor displays a SQL script with the following content:

```
109
110
111
112
113
114
115
116
117
118
119
120
121
```

The script includes a query:

```
-- Check Employees
SELECT * FROM Employees;
Scenario 3
-- Add New Customer
```

The output pane shows the results of the execution:

Execution time: 0.008 seconds

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1		1	1 Savings	500	6/20/2026, 7:...
2		2	2 Checking	2000	6/20/2026, 7:...

Supersetid :8003253

Exercise 3: Stored Procedures

```
SET SERVEROUTPUT ON;
```

```
-----  
-- Scenario 1  
-- Process Monthly Interest (1% for Savings Accounts)  
-----
```

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest  
IS  
BEGIN  
    UPDATE Accounts  
    SET Balance = Balance + (Balance * 0.01)  
    WHERE AccountType = 'Savings';  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Monthly Interest Applied Successfully');  
END;  
/
```

```
-- Test Scenario 1  
BEGIN  
    ProcessMonthlyInterest;  
END;  
/
```

```
SELECT * FROM Accounts;
```

```
-- Scenario 2  
-- Update Employee Bonus
```

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    dept VARCHAR2,  
    bonus NUMBER  
)  
IS  
BEGIN  
    UPDATE Employees
```

Supersetid :8003253

SET Salary = Salary + (Salary * bonus / 100)

WHERE Department = dept;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Bonus Updated Successfully');

END;

/

-- Test Scenario 2

BEGIN

UpdateEmployeeBonus('IT',10);

END;

/

SELECT * FROM Employees;

-- Scenario 3

-- Transfer Fund

CREATE OR REPLACE PROCEDURE TransferFunds(fromAccount NUMBER,toAccount NUMBER,amount
NUMBER)

IS

balance NUMBER;

BEGIN

SELECT Balance

INTO balance

FROM Accounts

WHERE AccountID = fromAccount;

IF balance >= amount THEN

UPDATE Accounts

SET Balance = Balance - amount

WHERE AccountID = fromAccount;

UPDATE AccountsSET Balance = Balance + amountWHERE AccountID = toAccount;

COMMIT; DBMS_OUTPUT.PUT_LINE('Amount Transferred Successfully');

ELSE

DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

END IF;

END;

/

Supersetid :8003253

-- Test Scenario 3

BEGIN

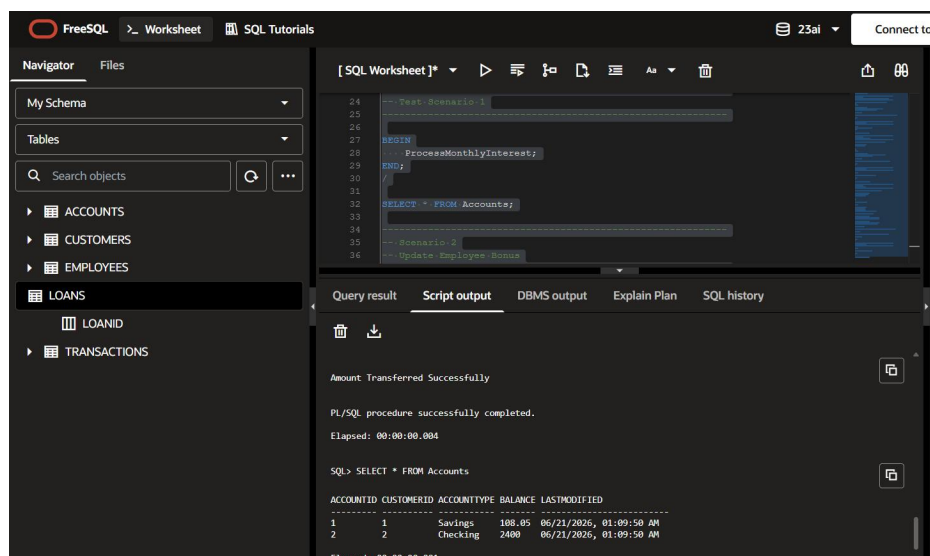
TransferFunds(1,2,200);

END;

/

SELECT * FROM Accounts;

OUTPUT:



Exercise 4: Functions

SET SERVEROUTPUT ON;

-- Scenario 1

CREATE OR REPLACE FUNCTION CalculateAge(

dob DATE

)

RETURN NUMBER

IS

age NUMBER;

Supersetid :8003253

BEGIN

age := TRUNC(MONTHS_BETWEEN(SYSDATE, dob) / 12);

RETURN age;

END;

/

-- Test Scenario 1

SELECT Name,

 CalculateAge(DOB) AS Age

FROM Customers;

-- Scenario 2

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(m

 loanAmount NUMBER,

 interestRate NUMBER,

 years NUMBER

)

RETURN NUMBER

IS

 monthlyInstallment NUMBER;

BEGIN

 monthlyInstallment :=

 (loanAmount + (loanAmount * interestRate * years / 100))

 /(years*12);

 RETURN monthl

-- Test Scenario 2

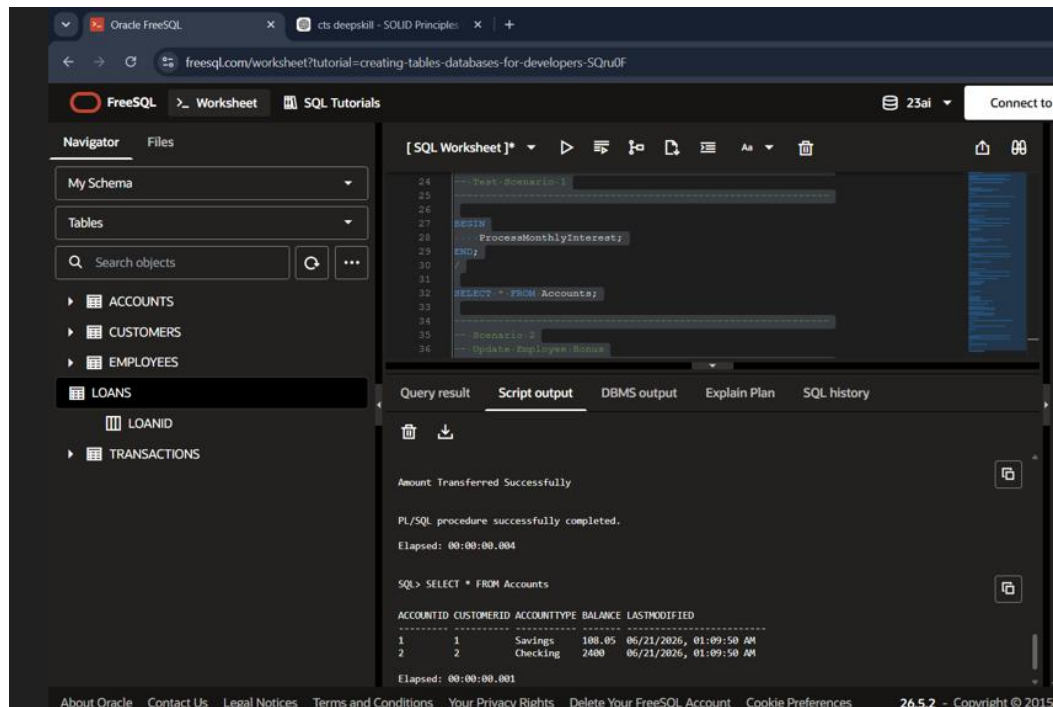
SELECT LoanID,

 CalculateMonthlyInstallment(LoanAmount,InterestRate,5)

Supersetid :8003253

```
AS Monthly_Installment
FROM Loans;
-- Scenario 3
CREATE OR REPLACE FUNCTION HasSufficientBalance(
    accId NUMBER,
    amount NUMBER
)
RETURN BOOLEAN
IS
    balance NUMBER;
BEGIN
    SELECT Balance
    INTO balance
    FROM Accounts
    WHERE AccountID = accId;
    IF balance >= amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END;
/
-- Test Scenario 3
DECLARE
    result BOOLEAN;
BEGIN
    result := HasSufficientBalance(1,500);
    IF result THEN
        DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
```

OUTPUT:



Exercises 5 Trigger

SET SERVEROUTPUT ON;

-- Scenario 1

CREATE OR REPLACE TRIGGER UpdateCustomerLastModified

BEFORE UPDATE

ON Customers

FOR EACH ROW

BEGIN

:NEW.LastModified := SYSDATE;

END;

/

-- Test Scenario 1

UPDATE Customers

SET Balance = 5000

WHERE CustomerID = 1;

COMMIT;

Supersetid :8003253

SELECT * FROM Customers;

-- Scenario 2

-- Create AuditLog Table

```
CREATE TABLE AuditLog(  
    LogID NUMBER GENERATED BY DEFAULT AS IDENTITY,  
    TransactionID NUMBER,  
    AccountID NUMBER,  
    Amount NUMBER,  
    ActionDate DATE  
);
```

-- Trigger to Log Transactions

CREATE OR REPLACE TRIGGER LogTransaction

AFTER INSERT

ON Transactions

FOR EACH ROW

BEGIN

```
    INSERT INTO AuditLog  
    (TransactionID,AccountID,Amount,ActionDate)  
    VALUES  
    (:NEW.TransactionID,  
     :NEW.AccountID,  
     :NEW.Amount,  
     SYSDATE);
```

END;

/

-- Test Scenario 2

INSERT INTO Transactions

VALUES

(

3,

1,

SYSDATE,

Supersetid :8003253

```
500,
'Deposit'
);
COMMIT;
SELECT * FROM AuditLog;
-- Scenario 3
-- Check Deposit and Withdrawal Rules
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT
ON Transactions
FOR EACH ROW
DECLARE
    balance NUMBER;
BEGIN
    SELECT Balance
    INTO balance
    FROM Accounts
    WHERE AccountID = :NEW.AccountID;
    IF :NEW.TransactionType='Withdrawal'
        AND :NEW.Amount > balance THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Insufficient Balance');
    END IF;
    IF :NEW.TransactionType='Deposit'
        AND :NEW.Amount<=0 THEN

        RAISE_APPLICATION_ERROR(
            -20002,
            'Deposit Amount Must Be Positive');
    END IF;
END;
```


Supersetid :8003253

/

```
INSERT INTO Transactions
```

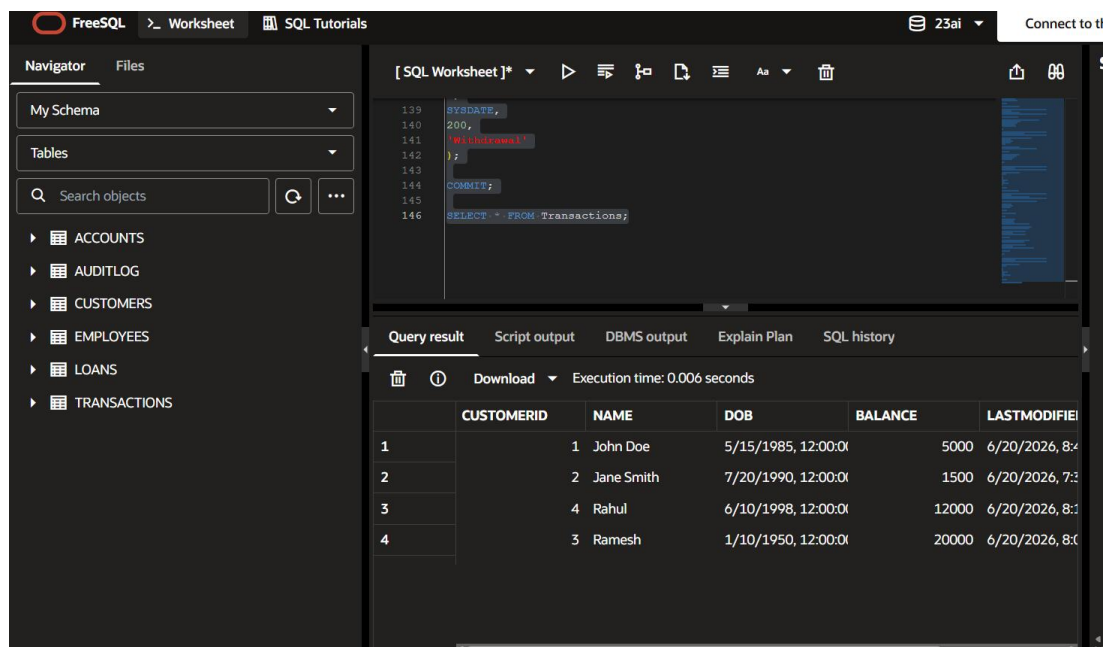
```
VALUES
```

```
(4,1,SYSDATE,200,'Withdrawal');
```

```
COMMIT;
```

```
SELECT * FROM Trans;
```

OUTPUT:



The screenshot shows the FreeSQL SQL editor interface. The left sidebar contains a 'Navigator' panel with 'My Schema' and 'Tables' sections. The 'Tables' section lists: ACCOUNTS, AUDITLOG, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main editor area displays the following SQL code:

```
139 SYSDATE,  
140 200,  
141 'Withdrawal',  
142 );  
143  
144 COMMIT;  
145  
146 SELECT * FROM Transactions;
```

Below the editor, the 'Query result' tab is active, showing the execution time as 0.006 seconds. The results are displayed in a table with the following columns: CUSTOMERID, NAME, DOB, BALANCE, and LASTMODIFIED. The table contains 4 rows of data:

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED
1	1	John Doe	5/15/1985, 12:00:00	5000	6/20/2026, 8:00:00
2	2	Jane Smith	7/20/1990, 12:00:00	1500	6/20/2026, 7:00:00
3	4	Rahul	6/10/1998, 12:00:00	12000	6/20/2026, 8:00:00
4	3	Ramesh	1/10/1950, 12:00:00	20000	6/20/2026, 8:00:00

Exercises 6 cursors

```
SET SERVEROUTPUT ON;
```

```
-- Scenario 1: Generate Monthly Statements
```

```
DECLARE
```

```
CURSOR GenerateMonthlyStatements IS
```

```
SELECT TransactionID,
```

```
AccountID,
```

```
Amount,
```

```
TransactionType
```

```
FROM Transactions
```

Supersetid :8003253

WHERE EXTRACT(MONTH FROM TransactionDate)=EXTRACT(MONTH FROM SYSDATE)

AND EXTRACT(YEAR FROM TransactionDate)=EXTRACT(YEAR FROM SYSDATE);

t GenerateMonthlyStatements%ROWTYPE;

BEGIN

OPEN GenerateMonthlyStatements;

LOOP

FETCH GenerateMonthlyStatements INTO t;

EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(

'Transaction ID : ' || t.TransactionID ||

' Account : ' || t.AccountID ||

' Amount : ' || t.Amount ||

' Type : ' || t.TransactionType);

END LOOP;

CLOSE GenerateMonthlyStatements;

END;

/

-- Scenario 2: Apply Annual Fee

DECLARE

CURSOR ApplyAnnualFee IS

SELECT AccountID

FROM Accounts;

BEGIN

FOR acc IN ApplyAnnualFee LOOP

UPDATE Accounts

SET Balance = Balance - 100

WHERE AccountID = acc.AccountID;

END LOOP;

Supersetid :8003253

```
COMMIT;

DBMS_OUTPUT.PUT_LINE('Annual Fee Applied');

END;

/SELECT * FROM Accounts;

-- Scenario 3: Update Loan Interest Rates

DECLARE

    CURSOR UpdateLoanInterestRates IS

        SELECT LoanID FROM Loans;

BEGIN

    FOR loan IN UpdateLoanInterestRates LOOP

        UPDATE Loans

        SET InterestRate = InterestRate + 0.5

        WHERE LoanID = loan.LoanID;

    END LOOP;

    COMMIT;

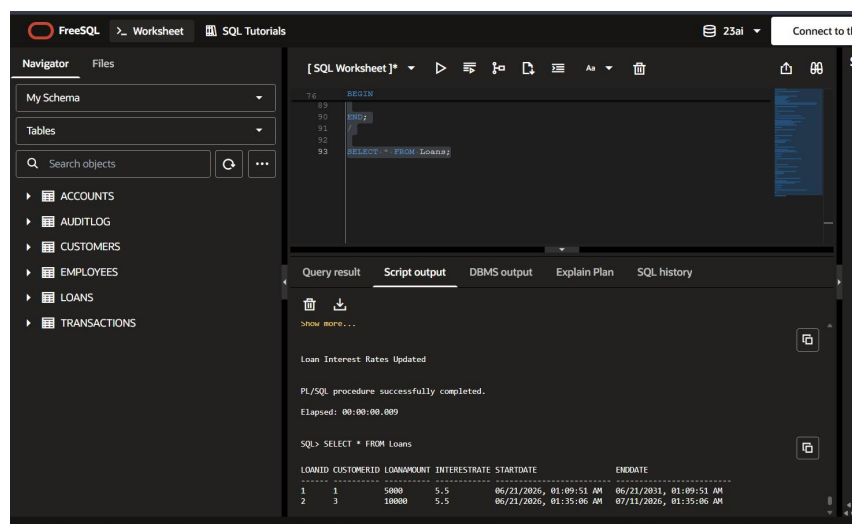
    DBMS_OUTPUT.PUT_LINE('Loan Interest Rates Updated');

END;

/

SELECT * FROM Loans;

OUTPUT:
```



Supersetid :8003253

Exercises 7 Packages

SET SERVEROUTPUT ON;

-- Scenario 1: Customer Management Package

CREATE OR REPLACE PACKAGE CustomerManagement AS

 PROCEDURE AddCustomer(

 cid NUMBER,

 cname VARCHAR2,

 dob DATE,

 bal NUMBER

);

 PROCEDURE UpdateCustomer(

 cid NUMBER,

 newBalance NUMBER

);

 FUNCTION GetBalance(

 cid NUMBER

) RETURN NUMBER;

END CustomerManagement;

/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

 PROCEDURE AddCustomer(

 cid NUMBER,

 cname VARCHAR2,

 dob DATE,

 bal NUMBER

)

IS BEGIN

 INSERT INTO Customers

 (CustomerID,Name,DOB,Balance,LastModified,IsVIP)

VALUES

 (cid,cname,dob,bal,SYSDATE,NULL);

COMMIT;

Supersetid :8003253

END;

PROCEDURE UpdateCustomer(

 cid NUMBER, newBalance NUMBER)

IS BEGIN

 UPDATE Customers

 SET Balance = newBalance

 WHERE CustomerID = cid;

COMMIT;

END;

FUNCTION GetBalance(cid NUMBER)

RETURN NUMBER IS

 bal NUMBER;

BEGIN

 SELECT Balance

 INTO bal

 FROM Customers

 WHERE CustomerID = cid;

 RETURN bal;

END;

END CustomerManagement;

/

BEGIN

 CustomerManagement.AddCustomer(5,'Arun', TO_DATE('1999-05-10','YYYY-MM-DD'), 5000);

 CustomerManagement.UpdateCustomer(5,8000);

END;

/

SELECT CustomerManagement.GetBalance(5) AS Balance

FROM Dual;

-- Scenario 2: Employee Management Package

CREATE OR REPLACE PACKAGE EmployeeManagement AS

 PROCEDURE HireEmployee(

 id NUMBER, name VARCHAR2, pos VARCHAR2, sal NUMBER, dept VARCHAR2

Supersetid :8003253

```
); PROCEDURE UpdateEmployee(  
    id NUMBER, newsalary NUMBER  
);  
  
FUNCTION AnnualSalary(  
    id NUMBER  
    ) RETURN NUMBER;  
END EmployeeManagement;  
  
/  
  
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS  
  
    PROCEDURE HireEmployee(  
        id NUMBER,  
        name VARCHAR2,  
        pos VARCHAR2,  
        sal NUMBER,  
        dept VARCHAR2  
    )  
  
    IS BEGIN  
        INSERT INTO Employees  
        VALUES(id,name, pos, sal, dept, SYSDATE);  
COMMIT;  
END;  
  
    PROCEDURE UpdateEmployee(  
        id NUMBER,  
        newsalary NUMBER )  
  
    IS BEGIN  
        UPDATE Employees  
        SET Salary = newsalary  
        WHERE EmployeeID=id;  
        COMMIT;  
END;  
  
    FUNCTION AnnualSalary( id NUMBER )  
  
    RETURN NUMBER
```

Supersetid :8003253

IS

sal NUMBER;

BEGIN

SELECT Salary

INTO sal

FROM Employees

WHERE EmployeeID=id;

RETURN sal*12;

END;

END EmployeeManagement;

/

BEGIN

EmployeeManagement.HireEmployee(

3,'David','Tester', 40000, 'IT'); EmployeeManagement.UpdateEmployee(3,45000);

END;

/

SELECT EmployeeManagement.AnnualSalary(3)

AS AnnualSalary

FROM Dual;

-- Scenario 3: Account Operations Package

CREATE OR REPLACE PACKAGE AccountOperations AS

PROCEDURE OpenAccount(

aid NUMBER, cid NUMBER,type VARCHAR2,bal NUMBER

);

PROCEDURE CloseAccount(

aid NUMBER

);

FUNCTION TotalBalance(cid NUMBER)

RETURN NUMBER;

END AccountOperations;

/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

Supersetid :8003253

```
PROCEDURE OpenAccount(  
    aid NUMBER,  
    cid NUMBER,  
    type VARCHAR2,  
    bal NUMBER  
)  
IS BEGIN  
    INSERT INTO Accounts  
    VALUES( aid,cid, type,bal, SYSDATE);  
COMMIT;  
END;  
PROCEDURE CloseAccount( aid NUMBER )  
IS  
BEGIN  
    DELETE FROM Accounts  
    WHERE AccountID=aid;  
    COMMIT;  
END;  
FUNCTION TotalBalance( cid NUMBER )  
RETURN NUMBER  
IS  
    total NUMBER;  
BEGIN  
    SELECT SUM(Balance)  
    INTO total  
    FROM Accounts  
    WHERE CustomerID=cid;  
    RETURN total;  
END;  
END AccountOperations;  
/  
BEGIN
```


Supersetid :8003253

AccountOperations.OpenAccount(

5,

1,

'Savings',

3000);

END;

/

SELECT AccountOperations.TotalBalance(1)

AS TotalBalance

FROM Dual;

BEGIN

AccountOperations.CloseAccount(5);

END;

/

SELECT * FROM Accounts;

OUTPUT:

The screenshot displays the FreeSQL SQL client interface. The left sidebar shows a 'Navigator' with 'My Schema' and 'Package' sections. The main editor shows a SQL script with line numbers 218 to 272. The script includes a PL/SQL procedure definition for 'AccountOperations' and its execution. The 'Query result' tab is active, showing the output of the script. The output indicates that the PL/SQL procedure was successfully completed and that 2 rows were selected from the 'Accounts' table.

SQL> BEGIN

AccountOperations.CloseAccount(5);

...

Show more...

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.009

SQL> SELECT * FROM Accounts

ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1	1	Savings	8.05	06/21/2026, 01:09:50 AM
2	2	Checking	2300	06/21/2026, 01:09:50 AM

Elapsed: 00:00:00.001

2 rows selected.